1 Advanced Encryption Standard (AES)

Why DES had to be replaced. By the late 1990s the 56-bit DES key was within reach of brute force: a 2^{56} search is about 7.2×10^{16} trials, and a purpose-built machine (EFF, 1998) found a DES key in a few days. Triple DES fixes the key length but runs three DES encryptions per block, which is slow in software, and it keeps DES's 64-bit block, which is too small for encrypting large volumes of data under one key. NIST therefore ran an open competition, judged on three criteria (*security, cost, implementation*), and in 2000 chose **Rijndael** by Daemen and Rijmen. It was published as the **Advanced Encryption Standard** (FIPS 197) in December 2001.

What kind of cipher it is. AES is a symmetric-key, non-Feistel block cipher that encrypts 128-bit blocks with a 128-, 192- or 256-bit key in 10, 12 or 14 rounds. Non-Feistel means that, unlike DES, the round does not split the block into halves and hide a one-way function inside a swap. Instead every step of the round is itself invertible, and decryption runs the inverse steps in reverse order. Such a design is called a substitution–permutation network: substitution (SubBytes) gives confusion, and permutation plus mixing (ShiftRows, MixColumns) give diffusion.

1.1 Parameters and data units

The block is always 128 bits and every round key is always 128 bits, whatever the cipher-key length. Only the number of rounds N_r depends on the key size, because a longer key needs more rounds before every key bit has influenced every state bit. Since one round key is added before round 1 and one in each of the N_r rounds, the number of round keys is $N_r + 1$, and each round key is 4 words.

Version	Key (bits)	Rounds N_r	Round keys $N_r + 1$	Key words $4(N_r + 1)$	Brute force
AES-128	128	10	11	44 (w_0–w_{43})	2^{128}
AES-192	192	12	13	52 (w_0–w_{51})	2^{192}
AES-256	256	14	15	60 (w_0–w_{59})	2^{256}

Data units. AES names five units. A *bit* is a binary digit; a *byte* is 8 bits and is treated as an element of GF(2^8), written as two hex digits; a *word* is 4 bytes (32 bits), seen either as a row or as a column of 4 bytes; a *block* is 16 bytes (128 bits), $b_0 b_1 \dots b_{15}$; and the *state* is the block arranged as a 4×4 matrix of bytes $s_{r,c}$, which is what every transformation actually acts on. Each column of the state is one word, so the state is also four words $w_0 w_1 w_2 w_3$ side by side.

Block to state. The 16 bytes fill the state **column by column**: bytes $b_0..b_3$ form column 0 (top to bottom), $b_4..b_7$ column 1, and so on, so that

$$s_{r,c} = b_{r+4c}, \qquad \text{equivalently} \qquad b_i \to s_{i \bmod 4,\, \lfloor i/4 \rfloor}.$$

The reverse mapping reads the state column by column to get the output block. This column-major order matters because MixColumns and AddRoundKey work on columns, so four consecutive input bytes are processed together.

Trap: filling the state row-wise

Writing the 16 bytes *across* the rows is the most common slip. The first four bytes go *down* column 0. With the wrong order every later step (ShiftRows, MixColumns, the key words as columns) gives a wrong answer.

WORKED EXAMPLE : text block to state (slide Example 1, from Forouzan)

Q. Show the state for the text block "AES uses a matrix", letters coded A = 00 to Z = 25 in hex, padding with Z.

A. Removing spaces gives 14 letters, so two bogus Z's make 16: AESUSESAMATRIXZZ. Each letter's number is written in **hex** (for example M = 12 = 0C, R = 17 = 11, X = 23 = 17, Z = 25 = 19):

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hex	00	04	12	14	12	04	12	00	0C	00	13	11	08	17	19	19

Filling column by column (first four bytes down column 0):

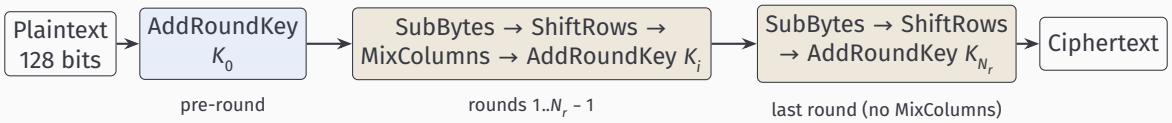
$$\text{State} = \begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix}$$

Check: $s_{1,3} = b_{1+12} = b_{13} = 17$ (the X). The slide prints 23 here; see the corrections box.

1.2 Round structure

Encryption of AES-128 is one key addition followed by 10 rounds:

1. **Pre-round:** AddRoundKey with round key $K_0 = w_0..w_3$ (the cipher key itself).
2. **Rounds 1 to** $N_r - 1$**:** SubBytes, ShiftRows, MixColumns, AddRoundKey (with $K_i = w_{4i}..w_{4i+3}$).
3. **Last round** N_r**:** SubBytes, ShiftRows, AddRoundKey, **without MixColumns**.



Why a key addition comes first. Every step except AddRoundKey is public and keyless. If the cipher began with SubBytes, an attacker who knows the plaintext could compute that step himself, so it would add work but no secrecy. Adding the key first (key whitening) means that from the very first non-linear step onward, nothing is computable without the key.

Why the last round has no MixColumns. MixColumns is linear over GF(2^8), so $\mathrm{MC}(Y) \oplus K = \mathrm{MC}\big(Y \oplus \mathrm{MC}^{-1}(K)\big)$. If the last round ended with MixColumns and then AddRoundKey, an attacker could apply InvMixColumns to the ciphertext and obtain $Y \oplus \mathrm{MC}^{-1}(K_{N_r})$, which is just the same cipher without the final MixColumns under a different (equally unknown) key. The final MixColumns therefore adds no security. Leaving it out costs nothing and makes the decryption algorithm have the same shape as encryption, which allows the "equivalent inverse cipher" to reuse the same hardware.

★ MUST KNOW

AES: block 128, key 128/192/256, rounds 10/12/14, round keys $N_r + 1$ (always 128 bits each), $4(N_r + 1)$ words. Pre-round AddRoundKey; rounds $1..N_r - 1$: **SubBytes, ShiftRows, MixColumns, AddRoundKey**; last round drops MixColumns. The state is filled **column by column**, $s_{r,c} = b_{r+4c}$.

1.3 SubBytes (substitution) ★

▸ **Likely this year:** "Find SubBytes of 0x53" or "apply SubBytes to the given state using the S-box"; or derive one S-box entry from the GF(2^8) inverse and the affine transform.

Problem and idea. A cipher built only from XORs, shifts and matrix multiplications is linear, and a linear cipher falls to solving a system of equations. SubBytes is the one non-linear step in AES. Each of the 16 bytes is replaced independently by looking it up in one fixed 16 × 16 table, the S-box. Because the same table is used for every byte, equal input bytes always give equal output bytes; SubBytes is an *intrabyte* transformation, since a byte's output depends on that byte alone.

Lookup rule. Write the byte as two hex digits *xy*. The **left digit** *x* **selects the row** and the **right digit** *y* **selects the column**; the entry there is the output. For example 53: row 5, column 3 gives ED, and 00: row 0, column 0 gives 63.

x\y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

AES S-box (FIPS-197), generated by script from the definition below and checked entry by entry.

How the S-box is built. The table is not arbitrary; each entry is computed in two steps from the byte *a* viewed as a polynomial in GF(2^8) with modulus $m(x) = x^8 + x^4 + x^3 + x + 1$ (11B).

1. **Multiplicative inverse:** $b = a^{-1}$ in GF(2^8), with 00 mapped to 00 (it has no inverse). The inverse map is highly non-linear, which is what resists linear and differential cryptanalysis. This is why AES works in GF(2^8) and not in $\mathbb{Z}_{256}$: in $\mathbb{Z}_{256}$ every even byte lacks an inverse, so the map would not be a permutation.

2. **Affine transformation over GF(2):** with $c = 63 = 0110\,0011$,

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i, \qquad i = 0..7.$$

This step breaks the simple algebraic form $a \mapsto a^{-1}$ and removes its fixed points ($00 \mapsto 00$ and $01 \mapsto 01$). As a result the AES S-box has no fixed point ($S(a) \neq a$ for all 256 bytes) and none with $S(a) = \bar{a}$.

InvSubBytes uses the inverse table: undo the affine map, then take the GF(2^8) inverse. Without that table, find the output in the S-box and read off its row and column (for example ED sits at row 5, column 3, so InvSubBytes(ED) = 53).

WORKED EXAMPLE : one S-box entry from the definition

Q. Show that SubBytes(53) = ED by computing it, not by looking it up.

A. *Step 1.* $53 = x^6 + x^4 + x + 1$. Its inverse modulo $m(x)$, by the extended Euclidean algorithm in GF(2^8), is $CA = x^7 + x^6 + x^3 + x$ (check: $53 \cdot CA = 01$). So $b = CA = 1100\,1010$, that is $b_7..b_0 = 1, 1, 0, 0, 1, 0, 1, 0$.
Step 2. Apply $b'_i = b_i \oplus b_{i+4} \oplus b_{i+5} \oplus b_{i+6} \oplus b_{i+7} \oplus c_i$ (indices mod 8, $c = 63$, so $c_0 = c_1 = c_5 = c_6 = 1$):

i	b_i	b_{i+4}	b_{i+5}	b_{i+6}	b_{i+7}	c_i	b'_i
0	0	0	0	1	1	1	**1**
1	1	0	1	1	0	1	**0**
2	0	1	1	0	1	0	**1**
3	1	1	0	1	0	0	**1**
4	0	0	1	0	1	0	**0**
5	0	1	0	1	0	1	**1**
6	1	0	1	0	0	1	**1**
7	1	1	0	0	1	0	**1**

Reading $b'_7..b'_0 = 1110\,1101$ gives **SubBytes(53) = ED**, which matches row 5, column 3 of the table.

WORKED EXAMPLE : SubBytes of a state (slide Example 2)

Q. Apply SubBytes to the state of Example 1.

A. Each byte is looked up independently (row = left digit, column = right digit): 00→63, 04→F2, 12→C9, 14→FA, 0C→FE, 13→7D, 11→82, 08→30, 17→F0, 19→D4.

$$\begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix} \xrightarrow{\text{SubBytes}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & \mathbf{F0} \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix}$$

The two 04's in row 1 both become F2, showing that SubBytes alone does not hide repeated bytes; ShiftRows and MixColumns are needed for that.

Single-bit changes spread: 5A and 5B differ in one bit, but $S(5A) = BE$ and $S(5B) = 39$ differ in four bits ($BE \oplus 39 = 87 = 1000\,0111$).

1.4 ShiftRows (permutation) ★

★ **In the papers:** 2025–26 Q4(a) (part of 6 marks): ShiftRows on a given state. Solved in full after AddRoundKey below.

Problem and idea. SubBytes changes each byte on its own, and MixColumns (next) mixes only the four bytes inside one column. Without a step that moves bytes *between* columns, the four columns would never interact, and AES would be four independent 32-bit ciphers running side by side, each attackable separately. ShiftRows is that step: it permutes bytes so that each column's four bytes end up in four different columns.

Mechanism. Row r of the state is rotated cyclically left by r bytes: row 0 is unchanged, row 1 moves left by 1, row 2 by 2, row 3 by 3. In symbols $s'_{r,c} = s_{r,(c+r) \bmod 4}$. The permutation is at the *byte* level: bytes change position but the bits inside a byte are untouched, unlike DES, whose P-boxes permute single bits. Since the four bytes of input column c lie in rows 0, 1, 2, 3, they are shifted by 0, 1, 2, 3 places and land in four different columns; after the next MixColumns every output column then depends on all four input columns. InvShiftRows rotates row r *right* by r bytes.

$$\begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{11} & s_{12} & s_{13} & s_{10} \\ s_{22} & s_{23} & s_{20} & s_{21} \\ s_{33} & s_{30} & s_{31} & s_{32} \end{pmatrix}$$

Applied to the SubBytes output of Example 2 (slide Example 3):

$$\begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & F0 \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & 63 & F0 & F2 \\ 7D & D4 & C9 & C9 \\ D4 & FA & 63 & 82 \end{pmatrix}$$

1.5 MixColumns (mixing) ★

▸ **Likely this year:** "Perform MixColumns on one column" (one or all four output bytes), or "show that the given column maps to …".

Problem and idea. SubBytes is intrabyte and ShiftRows only exchanges bytes, so neither makes a byte depend on its neighbours. MixColumns is the *interbyte* step: it replaces each column by a combination of all four of its bytes, so that changing one input byte changes all four output bytes. To guarantee that each new byte differs from the others even when all four inputs are equal, each input is first multiplied by a different constant and then the products are added. This is a matrix multiplication over GF(2^8), where "add" is XOR and "multiply" is polynomial multiplication modulo 11B:

$$\begin{pmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix}, \qquad \text{InvMixColumns: } \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}.$$

Each row of the matrix is the row above rotated right by one. Written out, $s'_0 = 02 \cdot s_0 \oplus 03 \cdot s_1 \oplus s_2 \oplus s_3$, $s'_1 = s_0 \oplus 02 \cdot s_1 \oplus 03 \cdot s_2 \oplus s_3$, $s'_2 = s_0 \oplus s_1 \oplus 02 \cdot s_2 \oplus 03 \cdot s_3$, $s'_3 = 03 \cdot s_0 \oplus s_1 \oplus s_2 \oplus 02 \cdot s_3$. The two matrices multiply to the identity in GF(2^8) (checked by script), which is why InvMixColumns undoes MixColumns.

Why the constants 01, 02, 03. They make encryption cheap, because multiplying by these needs only a shift and XORs. Multiplying by 02 is multiplying by x: shift the byte left by one bit, and if the bit shifted out (the old top bit b_7) was 1, the result has an x^8 term, which is reduced by XOR with 1B (since $x^8 \equiv x^4 + x^3 + x + 1$). This operation is called **xtime**. Multiplying by 03 = $x + 1$ is then $(02 \cdot b) \oplus b$. The matrix is also chosen so that the total number of non-zero bytes in an input column plus its output column is at least 5 (branch number 5), so one changed input byte always changes all four output bytes.

★ MUST KNOW : xtime

$02 \cdot b = (b \ll 1)$, then XOR 1B if the top bit of b was 1. $\quad 03 \cdot b = (02 \cdot b) \oplus b$. $\quad 01 \cdot b = b$.

WORKED EXAMPLE : one column (FIPS-197 test column)

Q. Apply MixColumns to the column $(DB, 13, 53, 45)^T$.

A. First the products by 02 and 03 of every input byte:

b	binary	shift left	top bit 1?	$02 \cdot b$ / $03 \cdot b = (02 \cdot b) \oplus b$
DB	1101 1011	1011 0110 = B6	yes: B6 ⊕ 1B	AD / 76
13	0001 0011	0010 0110 = 26	no	26 / 35
53	0101 0011	1010 0110 = A6	no	A6 / F5
45	0100 0101	1000 1010 = 8A	no	8A / CF

Then each output byte is a row of the matrix times the column, XORed left to right:

$$s'_0 = 02{\cdot}DB \oplus 03{\cdot}13 \oplus 53 \oplus 45 = AD \oplus 35 \oplus 53 \oplus 45 = 98 \oplus 53 \oplus 45 = CB \oplus 45 = \mathbf{8E}$$
$$s'_1 = DB \oplus 02{\cdot}13 \oplus 03{\cdot}53 \oplus 45 = DB \oplus 26 \oplus F5 \oplus 45 = FD \oplus F5 \oplus 45 = 08 \oplus 45 = \mathbf{4D}$$
$$s'_2 = DB \oplus 13 \oplus 02{\cdot}53 \oplus 03{\cdot}45 = DB \oplus 13 \oplus A6 \oplus CF = C8 \oplus A6 \oplus CF = 6E \oplus CF = \mathbf{A1}$$
$$s'_3 = 03{\cdot}DB \oplus 13 \oplus 53 \oplus 02{\cdot}45 = 76 \oplus 13 \oplus 53 \oplus 8A = 65 \oplus 53 \oplus 8A = 36 \oplus 8A = \mathbf{BC}$$

Answer: $(DB, 13, 53, 45) \to (8E, 4D, A1, BC)$, the FIPS-197 test value. Check: every output byte changed, as the branch-number argument predicts.

A second check comes from the exam state below: column 0 after ShiftRows is $(87, 6E, 46, A6)$. With $02 \cdot 87 = 0E \oplus 1B = 15$, $03 \cdot 6E = DC \oplus 6E = B2$, the first output byte is $15 \oplus B2 \oplus 46 \oplus A6 = 47$, and the whole column becomes $(47, 37, 94, ED)$, which is exactly column 0 of the state given in part (b) of that question (Stallings' figures chain ShiftRows → MixColumns → AddRoundKey).

1.6 AddRoundKey (key adding) ★

★ **In the papers:** 2025–26 Q4(b) (part of 6 marks): AddRoundKey of a given state with a given key matrix.

Problem and idea. The three steps above are fixed and public, so on their own they would be a keyless scrambling that anyone can undo. AddRoundKey is the only step that uses the key: it XORs the 128-bit round key into the state. It proceeds column by column, like MixColumns, but where MixColumns multiplies each column by a constant matrix, AddRoundKey *adds* a key word to it: in round i, column c becomes $s_c \oplus w_{4i+c}$. Equivalently, write the round key as a 4×4 matrix whose columns are the four words and XOR the two matrices entry by entry.

Inverse. Addition and subtraction in GF(2^8) are both XOR, and $x \oplus k \oplus k = x$, so AddRoundKey is its own inverse; decryption uses the same operation with the same round key.

Doing the XOR by hand. XOR one hex digit at a time, through its 4-bit binary form: $4 \oplus A = 0100 \oplus 1010 = 1110 = E$. Useful shortcuts: $x \oplus x = 0$, $x \oplus 0 = x$, $x \oplus F$ flips all four bits.

WORKED EXAMPLE : 2025–26 Q4 (6 marks), Stallings' AES example

Q. In AES, the state is given below. (a) Show the ShiftRows transformation. (b) After MixColumns, assume the state and the round key are as given. Perform AddRoundKey.

A. (a) Row r is rotated left by r bytes: row 0 unchanged; row 1 EC 6E 4C 90 → 6E 4C 90 EC; row 2 4A C3 46 E7 → 46 E7 4A C3; row 3 8C D8 95 A6 → A6 8C D8 95.

$$\begin{pmatrix} 87 & F2 & 4D & 97 \\ EC & 6E & 4C & 90 \\ 4A & C3 & 46 & E7 \\ 8C & D8 & 95 & A6 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 87 & F2 & 4D & 97 \\ \mathbf{6E} & \mathbf{4C} & \mathbf{90} & \mathbf{EC} \\ \mathbf{46} & \mathbf{E7} & \mathbf{4A} & \mathbf{C3} \\ \mathbf{A6} & \mathbf{8C} & \mathbf{D8} & \mathbf{95} \end{pmatrix}$$

(b) AddRoundKey is the byte-wise XOR of the state and the key matrix, $s'_{r,c} = s_{r,c} \oplus k_{r,c}$:

$$\begin{pmatrix} 47 & 40 & A3 & 4C \\ 37 & D4 & 70 & 9F \\ 94 & E4 & 3A & 42 \\ ED & A5 & A6 & BC \end{pmatrix} \oplus \begin{pmatrix} AC & 19 & 28 & 57 \\ 77 & FA & D1 & 5C \\ 66 & DC & 29 & 00 \\ F3 & 21 & 41 & 6A \end{pmatrix} = \begin{pmatrix} \mathbf{EB} & \mathbf{59} & \mathbf{8B} & \mathbf{1B} \\ \mathbf{40} & \mathbf{2E} & \mathbf{A1} & \mathbf{C3} \\ \mathbf{F2} & \mathbf{38} & \mathbf{13} & \mathbf{42} \\ \mathbf{1E} & \mathbf{84} & \mathbf{E7} & \mathbf{D6} \end{pmatrix}$$

Working, digit by digit (left digits, then right digits):

```
47⊕AC: 4⊕A=E, 7⊕C=B → EB   40⊕19: 5, 9 → 59   A3⊕28: 8, B → 8B   4C⊕57: 1, B → 1B
37⊕77: 4, 0 → 40            D4⊕FA: 2, E → 2E   70⊕D1: A, 1 → A1   9F⊕5C: C, 3 → C3
94⊕66: F, 2 → F2            E4⊕DC: 3, 8 → 38   3A⊕29: 1, 3 → 13   42⊕00: 4, 2 → 42
ED⊕F3: 1, E → 1E            A5⊕21: 8, 4 → 84   A6⊕41: E, 7 → E7   BC⊕6A: D, 6 → D6
```

For example $47 \oplus AC$: $0100\,0111 \oplus 1010\,1100 = 1110\,1011 = EB$. Check: XORing the answer with the key again returns the given state (for instance $EB \oplus AC = 47$), because AddRoundKey is self-inverse.

Trap: AddRoundKey

Do not add bytes as integers ($47 + AC = F3$ is wrong; the answer is $47 \oplus AC = EB$). And in the ShiftRows part, shift *left*, by the row index starting from 0; rotating right, or shifting row 1 by 2, is the usual slip.

1.7 Key expansion (AES-128) ★

▸ **Likely this year:** "Given the cipher key, find w_4–w_7 (the first round key)", Forouzan/Stallings style, using the S-box table.

Problem. AES-128 needs 11 round keys of 128 bits, that is 44 words $w_0..w_{43}$, but the user supplies only one 128-bit key. The expansion must be cheap, must spread every key bit into many round-key bits, and must not be linear; otherwise knowing part of one round key would give the others by simple algebra, and keys with patterns (like DES's weak keys) would produce repeating round keys.

Mechanism. The first four words are the cipher key itself, taken 4 bytes at a time: $w_0 = k_0k_1k_2k_3$, $w_1 = k_4..k_7$, and so on. Each later word is the word four positions back XORed with the previous word, except that every fourth word passes the previous word through a non-linear function first:

$$w_i = \begin{cases} w_{i-4} \oplus w_{i-1}, & i \bmod 4 \neq 0, \\ w_{i-4} \oplus t_i, \quad t_i = \text{SubWord}\big(\text{RotWord}(w_{i-1})\big) \oplus \text{RCon}[i/4], & i \bmod 4 = 0. \end{cases}$$

- **RotWord** rotates the 4 bytes of a word left by one: $[a_0, a_1, a_2, a_3] \to [a_1, a_2, a_3, a_0]$. It is ShiftRows applied to a single row.
- **SubWord** applies the S-box to each of the 4 bytes. It is SubBytes on a single word and supplies the non-linearity.
- **RCon** (round constant) is a word whose rightmost three bytes are zero and whose left byte is $x^{j-1} \bmod m(x)$ for round j. Each value is 02 times the previous one in GF(2^8), so 01, 02, 04, 08, 10, 20, 40, 80,

then $02 \cdot 80$ overflows and is reduced to $00 \oplus 1B = 1B$, and $02 \cdot 1B = 36$. RCon is different in every round, which breaks the symmetry between rounds, so that no key gives identical round keys.

Round j	1	2	3	4	5	6	7	8	9	10
RCon[j]	01	02	04	08	10	20	40	80	1B	36

(each followed by 00 00 00)

Round key K_j is the four words $w_{4j}, w_{4j+1}, w_{4j+2}, w_{4j+3}$, placed as the *columns* of a 4×4 matrix, so that AddRoundKey can XOR column c with w_{4j+c}. For AES-192 and AES-256 the rule is the same with 6 or 8 words per step instead of 4 (t is computed when $i \bmod 6 = 0$ or $i \bmod 8 = 0$); AES-256 additionally applies SubWord alone when $i \bmod 8 = 4$.

WORKED EXAMPLE : first round key (FIPS-197 / Stallings key)

Q. The AES-128 cipher key is 2B7E1516 28AED2A6 ABF71588 09CF4F3C. Find w_4, w_5, w_6, w_7.
A. w_0 = 2B7E1516, w_1 = 28AED2A6, w_2 = ABF71588, w_3 = 09CF4F3C.
$i = 4$ is a multiple of 4, so compute t_4 from w_3:

RotWord(w_3)	09 CF 4F 3C → CF 4F 3C 09
SubWord	S(CF) = 8A (row C, col F), S(4F) = 84, S(3C) = EB, S(09) = 01 ⇒ 8A84EB01
⊕ RCon[1]	8A84EB01 ⊕ 01000000 = t_4 = 8B84EB01

Then

$$w_4 = w_0 \oplus t_4 = 2B7E1516 \oplus 8B84EB01 = \mathbf{A0FAFE17}$$
$$w_5 = w_1 \oplus w_4 = 28AED2A6 \oplus A0FAFE17 = \mathbf{88542CB1}$$
$$w_6 = w_2 \oplus w_5 = ABF71588 \oplus 88542CB1 = \mathbf{23A33939}$$
$$w_7 = w_3 \oplus w_6 = 09CF4F3C \oplus 23A33939 = \mathbf{2A6C7605}$$

As a matrix (words as columns) the round-1 key is

$$K_1 = \begin{pmatrix} A0 & 88 & 23 & 2A \\ FA & 54 & A3 & 6C \\ FE & 2C & 39 & 76 \\ 17 & B1 & 39 & 05 \end{pmatrix}.$$

Check: these are the FIPS-197 Appendix A values; continuing the same way gives t_8 = 52386BE5 (RotWord(2A6C7605) = 6C76052A, SubWord = 50386BE5, RCon = 02), w_8 = F2C295F2, and finally w_{43} = B6630CA6.

1.8 One complete round: FIPS-197 example

Putting the four steps together on the standard test vector (plaintext 3243F6A8 885A308D 313198A2 E0370734, the key above) shows what a full round looks like; the mock paper's AES question (Q8) takes its matrices from this trace. The plaintext fills the state column by column, the pre-round XORs it with K_0 (the cipher key), and round 1 then gives:

$$\underbrace{\begin{pmatrix} 19 & A0 & 9A & E9 \\ 3D & F4 & C6 & F8 \\ E3 & E2 & 8D & 48 \\ BE & 2B & 2A & 08 \end{pmatrix}}_{\text{after } K_0} \xrightarrow{\text{Sub}} \begin{pmatrix} D4 & E0 & B8 & 1E \\ 27 & BF & B4 & 41 \\ 11 & 98 & 5D & 52 \\ AE & F1 & E5 & 30 \end{pmatrix} \xrightarrow{\text{Shift}} \begin{pmatrix} D4 & E0 & B8 & 1E \\ BF & B4 & 41 & 27 \\ 5D & 52 & 11 & 98 \\ 30 & AE & F1 & E5 \end{pmatrix} \xrightarrow{\text{Mix}} \begin{pmatrix} 04 & E0 & 48 & 28 \\ 66 & CB & F8 & 06 \\ 81 & 19 & D3 & 26 \\ E5 & 9A & 7A & 4C \end{pmatrix} \xrightarrow{\oplus K_1} \begin{pmatrix} A4 & 68 & 6B & 02 \\ 9C & 9F & 5B & 6A \\ 7F & 35 & EA & 50 \\ F2 & 2B & 43 & 49 \end{pmatrix}$$

After all ten rounds the ciphertext is 3925841D 02DC09FB DC118597 196A0B32; a Python AES implementation reproduces this and decrypts it back.

1.9 Decryption

Since every step is invertible, decryption applies the inverse steps in reverse order with the round keys in reverse order (K_{N_r} first, K_0 last). In the original design (Forouzan's figure) this gives:

1. AddRoundKey with $K_{N_r} = w_{40}..w_{43}$.
2. Each of the first $N_r - 1$ decryption rounds: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
3. Last decryption round: InvShiftRows, InvSubBytes, AddRoundKey with K_0 (no InvMixColumns).

InvShiftRows rotates right, InvSubBytes uses the inverse S-box, InvMixColumns uses the matrix [0E 0B 0D 09] and AddRoundKey is its own inverse. Because InvShiftRows and InvSubBytes commute (one moves bytes, the other changes each byte on its own) and InvMixColumns is linear, the steps can be reordered into the same

pattern as encryption (InvSubBytes, InvShiftRows, InvMixColumns, AddRoundKey) if the middle round keys are passed through InvMixColumns; this is the *equivalent inverse cipher*. Unlike DES, AES decryption is a different algorithm from encryption, and InvMixColumns, with its larger constants, is slower than MixColumns.

1.10 Analysis of AES

Brute force. A 128-bit key needs up to 2^{128} trials against DES's 2^{56}, so if DES can be broken in t seconds, AES-128 needs $2^{128}/2^{56} \cdot t = 2^{72}\,t$ seconds. The 192- and 256-bit versions widen the gap further. The best published shortcut (biclique attack, 2011) lowers the AES-128 cost only to about 2^{126}, which is still far out of reach.

No weak keys. DES has keys whose 16 round keys are all equal, so encrypting twice decrypts. In AES, SubWord makes the key schedule non-linear and RCon makes each round's step different, so no such keys are known.

Diffusion and statistical attacks. The steps work together to spread a single change over the whole block in two rounds. In round 1 a changed byte stays one byte through SubBytes and ShiftRows, then MixColumns spreads it to all 4 bytes of its column. In round 2 ShiftRows sends those 4 bytes to 4 different columns, and MixColumns spreads each of them over its column, so all 16 bytes have changed (checked by script: 4 bytes after one round, 16 after two). With 10 or more rounds the ciphertext shows no frequency pattern of the plaintext, and statistical analysis has failed.

Differential and linear cryptanalysis. AES was designed after these attacks were known; the S-box (GF(2^8) inverse) has low differential and linear probabilities, and the branch number of MixColumns forces many active S-boxes over a few rounds, so no such attack works on full AES.

Simplicity and cost. The steps are byte operations, table lookups and XORs, so AES is fast on cheap 8-bit processors with little memory as well as on 32-bit machines (where SubBytes, ShiftRows and MixColumns can be merged into table lookups), and many processors have AES instructions.

1.11 AES versus DES

	DES	AES
Structure	Feistel (round function need not be invertible)	non-Feistel SPN (every step invertible)
Block size	64 bits	128 bits
Key size	56 bits (64 with parity)	128, 192 or 256 bits
Rounds	16	10, 12 or 14
Round key	48 bits, 16 keys	128 bits, $N_r + 1$ keys
Works on	bits (expansion, P-boxes)	bytes in GF(2^8) (state matrix)
Non-linearity	eight 6-to-4-bit S-boxes	one 8-to-8-bit S-box (inverse + affine)
Permutation	bit-level P-boxes	byte-level ShiftRows
Decryption	same algorithm, keys reversed	inverse steps, keys reversed
Brute force	2^{56} (broken in practice)	2^{128} or more
Weak keys	4 weak, 12 semi-weak	none known

Corrections to the notes / slides

- **Example 1, letter X.** The slide writes X as 23. X is letter 23 in *decimal*, which is 17 in hex (the same slide correctly converts M = 12 → 0C and R = 17 → 11). With 17, SubBytes gives F0; the slide's 26 (in Examples 2 and 3) is $S(23)$. If the exam reuses the slide's state, say which value you used.
- **S-box, row B, column 1.** The slide's table prints C8; the correct FIPS-197 entry is $S(B1) = C8$. C8 already appears at row 5, column 9 ($S(59) = C8$), and an S-box is a permutation, so it cannot appear twice. The inverse table on the next slide is right ($\text{InvS}(C8) = B1$). All other 255 entries of the slide match.
- **ShiftRows rows.** The slide numbers the rows 1 to 4 ("Row 1: no shifting, Row 2: 1-byte left shift, ..."), while the pseudocode and these notes number them 0 to 3. Either way the top row is not shifted and the bottom row moves left by 3.

Check your understanding

1. **Why does AES do its arithmetic in GF(2^8) rather than in $\mathbb{Z}_{256}$?** The S-box needs the inverse of every non-zero byte and MixColumns needs an invertible matrix. In $\mathbb{Z}_{256}$ the 128 even bytes have no inverse, while GF(2^8) is a field, so every non-zero byte has one, and addition is plain XOR.
2. **How many rounds, round keys and key words does AES-192 use?** 12 rounds, 12 + 1 = 13 round keys of 128 bits each, so 13 × 4 = 52 words.
3. **What would happen if ShiftRows were removed?** Every other step works within one column, so the four columns would never mix. AES would become four independent 32-bit ciphers, each of which could be attacked separately with a 2^{32}-sized codebook.
4. **Why is AddRoundKey its own inverse, and why is it the only keyed step?** XOR with k twice gives back the input. The other steps are fixed public maps, so all secrecy must enter through AddRoundKey, which is why

it also comes before round 1.

5. **Compute** $02 \cdot \mathbf{87}$ **and** $03 \cdot \mathbf{87}$**.** 87 = 1000 0111; shifting gives 0000 1110 = 0E, and the top bit was 1, so $02 \cdot 87 = 0E \oplus 1B = 15$. Then $03 \cdot 87 = 15 \oplus 87 = 92$.
6. **What is SubBytes(00), and why is it not 00?** 00 has no inverse and is mapped to 00, but the affine step then XORs in the constant 63, so $S(00) = 63$. The constant is there precisely so that the S-box has no fixed points.
7. **Why is MixColumns omitted in the last round?** It is linear and keyless, so a final MixColumns could be undone by anyone applying InvMixColumns to the ciphertext (moving it past AddRoundKey only changes the key). It adds no security, and omitting it lets decryption have the same structure as encryption.
8. **Given** w_7 = **2A6C7605 and** w_4 = **A0FAFE17, find** w_8**.** 8 mod 4 = 0: RotWord gives 6C76052A, SubWord gives 50386BE5, XOR RCon[2] = 02000000 gives t_8 = 52386BE5, and $w_8 = w_4 \oplus t_8$ = F2C295F2.
9. **Is AES a Feistel cipher? Differentiate.** No. In a Feistel cipher (DES) half the block passes through a round function that need not be invertible and the halves are swapped, so decryption is the same algorithm. AES transforms the whole state each round with invertible steps, so decryption needs the inverse steps.
10. **After how many rounds does every output byte depend on every input byte?** Two: MixColumns in round 1 spreads one byte over its column, and ShiftRows plus MixColumns in round 2 spread those four bytes over all four columns.